Embedded IoT Quiz Prep

ESP32 basics, interrupts, hardware timers and button debouncing

CSE-3079 · Lecture 1, Week 3, Week 4, Git guide

ESP32 vs Arduino Uno (Lecture 1)

Faster and bigger

ESP32: 240 MHz, ~520 KB SRAM, 4-16 MB flash. Uno: 16 MHz, 2 KB RAM, 32 KB flash.

Dual core, 32-bit

Xtensa LX6. Core 0 runs Wi-Fi/radio, Core 1 runs your code. FreeRTOS shares the work.

Three kinds of memory

Flash = stores the program. SRAM = fast variables. ROM/RTC = bootloader; RTC stays on in deep sleep.

Wi-Fi + Bluetooth

Both use 2.4 GHz and one antenna. It switches between them very fast (time-division multiplexing), so they seem simultaneous.

Power, security, family, pins

Power modes

Active: 160-260 mA. Deep sleep: ~10 µA. A tiny ULP coprocessor can still check sensors.

Security

Hardware AES/SHA/RSA, Secure Boot (signed firmware only), Flash Encryption.

Family

S3 = AI/vision. C3/C6 = RISC-V, Wi-Fi 6. H2 = Zigbee/Thread, no Wi-Fi.

DevKit parts

AMS1117 turns 5 V into 3.3 V. CP2102/CH340 is the USB-to-UART chip used for flashing.

Pin rules

GPIO 34, 35, 36, 39 = input only. GPIO 6-11 = connected to flash, never use.

Strapping pins

GPIO 0, 2, 5, 12, 15 affect boot. Use with care.

Polling vs interrupts, and ISR rules (Week 3)

Polling

CPU keeps checking the pin. Wastes cycles and blocks other work (~99% CPU).

Interrupt

CPU does its own work. When the pin changes, the ISR runs, then returns (~1% CPU).

Button with INPUT_PULLUP

Resting = HIGH. Press = LOW. So a press is a FALLING edge. Modes: RISING, FALLING, CHANGE.

ISR rules

Keep it short. Mark shared variables volatile. No delay() or Serial.print(). Use IRAM_ATTR / ARDUINO_ISR_ATTR so code sits in RAM, not flash.

Attach example

```
pinMode(pin, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(pin),
isr, FALLING);
```

Best habit

ISR only sets a flag. loop() does the real work.

Switch bounce and debounce options

What is bounce?

Metal contacts vibrate for about 5-50 ms. One press makes many HIGH/LOW edges, so the ISR fires many times.

The ESP32 can't tell

It reacts to every edge. It sees electricity, not human intent.

Option 1: RC filter + Schmitt trigger

Works well but needs extra parts.

Option 2: delay(50)

Simple, but blocks the CPU. Bad in an ISR.

Option 3: timer-based (best)

Hardware timer measures the time. No blocking, no extra parts.

Hardware timers and the maths (Week 4)

What is it?

A peripheral that counts ticks by itself. At the alarm value it fires an interrupt. ESP32 has four 64-bit timers.

Frequency to tick

80 MHz APB clock, divided to 1 MHz. 1 tick = 1 µs.

Formulas

T = N / f and N = T × f. Example: 50 ms × 1,000,000 = 50,000 ticks.

Quick table

1 ms = 1,000 | 10 ms = 10,000 | 50 ms = 50,000 | 1 s = 1,000,000 ticks (at 1 MHz)

Periodic (auto-reload)

Repeats forever. Use for LED blink, sensor sampling.

One-shot

Fires once. Use for debounce validation.

Timer API: 3.x (current) vs 2.x (old)

Arduino-ESP32 3.x

```
timerBegin(1000000);
timerAttachInterrupt(t, &onTimer);
timerAlarm(t, 1000000, true, 0);
```

Arduino-ESP32 2.x (old)

```
timerBegin(0, 80, true);
timerAttachInterrupt(t, &onTimer, true);
timerAlarmWrite(t, 1000000, true);
timerAlarmEnable(t);
```

timerAlarm arguments

(timer, ticks, autoreload, reload_count). true = repeat, false = one-shot.

Lab 1: blink LED

The timer ISR toggles ledState and calls digitalWrite(LED_PIN, ledState) every 1 s. loop() stays empty.

Timer-based debounce: how it works

1. Trigger

First FALLING edge enters the GPIO ISR. If debounceActive is false, continue.

2. Arm

Set debounceActive = true. timerWrite(t, 0); timerAlarm(t, 50000, false, 0).

3. Wait

Bounce edges are ignored. loop() keeps running.

4. Validate

After 50 ms the timer ISR reads the pin. Still LOW = valid press, do the action.

5. Reset

debounceActive = false so the next press works.

Why it's good

Zero delay(), no wasted CPU, ignores noise. GPIO ISR asks 'did an edge occur?'. Timer ISR asks 'has 50 ms passed?'.

Quick quiz answers

Why not delay() in an ISR?

It blocks, breaking the short-ISR rule.

Why volatile?

Value can change inside an ISR, so the compiler must not cache it.

Why one-shot for debounce?

You validate once, not repeatedly.

Role of debounceActive?

Stops bounce edges from restarting the timer.

1 MHz means?

1 tick = 1 µs, so 30 ms = 30,000 ticks.

Pressed button reads?

LOW with INPUT_PULLUP.

Git and GitHub with PlatformIO

First push

```
git init
git add .
git commit -m "Initial"
git branch -M main
git remote add origin <URL>
git push -u origin main
```

Daily routine

git status, git add ., git commit -m "msg", git push

Key words

Git = version control on your PC. GitHub = online host. Commit = saved snapshot. origin = name of the remote.

Semester repo

One repo, one folder per lab (Lab01_Blink...). Put .pio/ and .vscode/ in .gitignore. Never commit Wi-Fi passwords.